

Lab F report

By Attila Garganego (22339628)

Abstract

In today's world we use electronics all the times to store information electronically and use such information for other tasks. In this lab the slave and master latches are tested and used to create counters and LFSR (Linear Feedback Shift Register) and clocks are created and implemented using Vivado and implemented using a Basys 3 FPGA board. The importance of synchronous circuits and of a synchronous reset, make circuits inputs more stable and reliable.

Introduction

In this lab report it was tested the importance of a synchronous circuit on a slave latch and how to connect up a clock and a reset to a circuit. It was also created and tested an 8-bit temp register that increased, decreased and set itself into 22 values according to the button pressed. Moreover, a linear feedback shifting register was created and a testbench to go along with it to make it function and test edge cases and what would happen if the forbidden seed value was used as the seed value for my LFSR. A counter in addition was created to see how many 1's and 0's passed as the MSB before the LFSR looped. The last thing done in this lab was to create a clock file that could create a clock speed desired of 1s to run to LFSR.

Methodology

The main methodology I will discuss in this report is of how I decided to create my testbenches and not how I coded my latches or other circuits.

Part A

For the testbench of part A (tb_d_type_ff.v) I tried to copy as closely as possible the waveform displayed on the pdf. When deciding how to make the signals synchronous I chose to lengthen the signals high up to where a clock cycle began.

Part B

For part B there were no testbenches to create.

Part C

For part C I had to create an LFSR of 18 tap length and I needed it to be an xnor gate. To comprehend better what bits are supposed to be tapped into an 18 tap length LFSR was drawn.

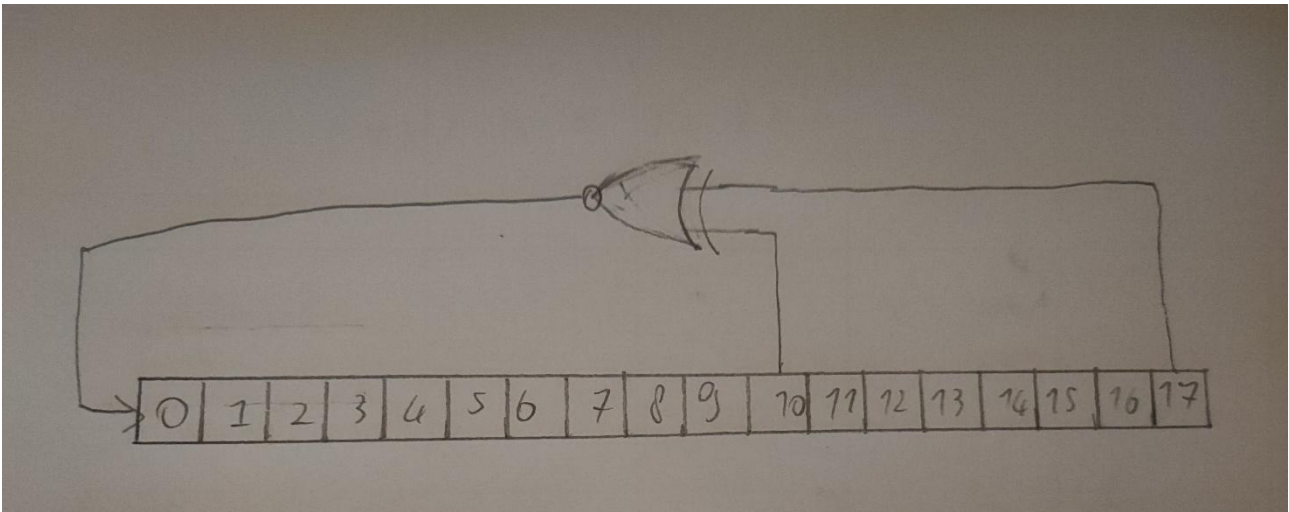


Figure 1: LFSR diagram of 18 tap length with xnor gate

I created the testbench (fig.1). It was set up like this to test the LFSR's functionality when operating in normal conditions and making sure that when the registry was reset the initial seed value was reset in the registry and if the max_reg value would reset back to 0 when required to.

```

initial begin
    re_set = 1'b1; //initialising reset to high
    enable = 1'b0; // initialising enable low
    #100 //pausing for 10 clock cycles.
    re_set = 1'b0; //reset is then set to low
    #10
    enable = 1'b1; //enable is set to high
    #2621440 // pausing for 2^n -1 bit shifts
    enable = 1'b0; // set to low
    re_set = 1'b1;
    #10;
    re_set = 1'b0; //reset is then set to low
    #10
    enable = 1'b1; //enable is set to high
    #2621440; // pausing for 2^n -1 bit shifts
    $stop; //stops simulation so it doesn't run onto infinity
end

```

Figure 2: Testbench for lab F part c

For the advanced part C the counter modules I/O ports were drawn to get an idea of what inputs and outputs the module would need to function.

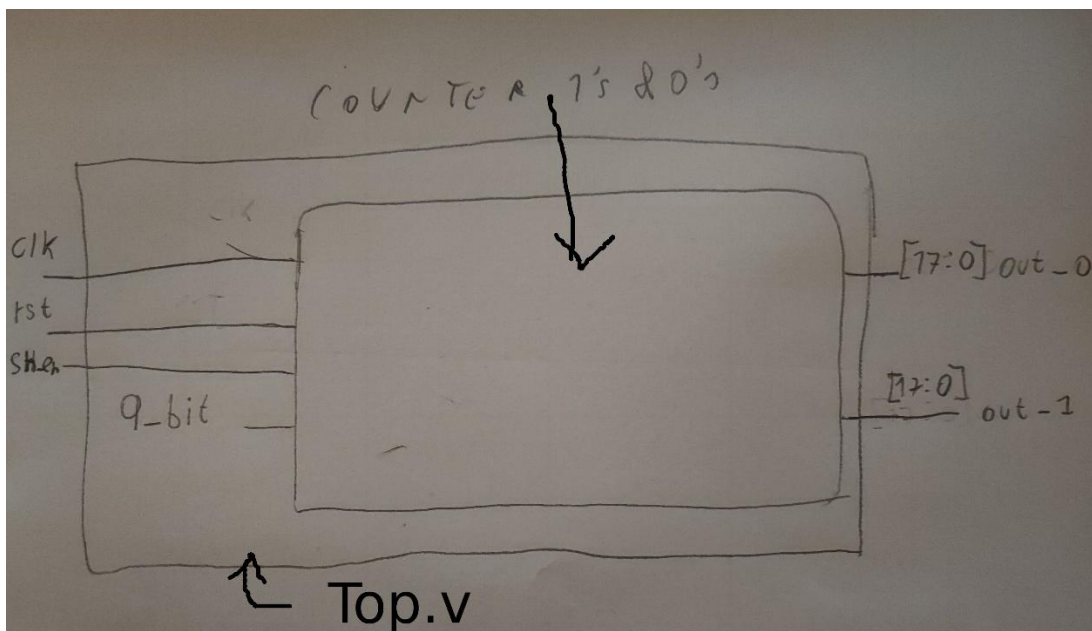


Figure 3: Counter of 1's and 0's I/O ports

Part D

In part D I had to make a few changes to my LFSR module. As I wasn't using the outputs from the counter, I removed it and moreover as my Basys 3 board doesn't have 18 LEDs, I had to make the first 9 bits be

displayed with a switch turned off and if the switch was turned on then the last 9 bits would be displayed. The max reg led would still be displayed whether the switch was on or off.

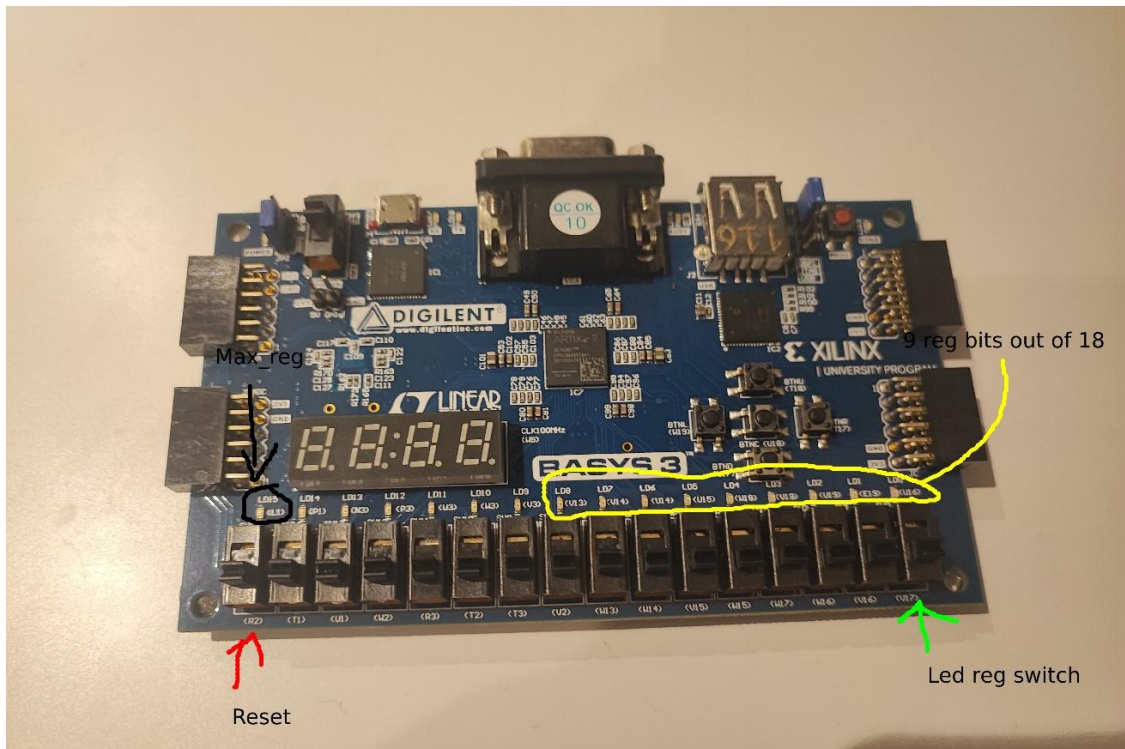


Figure 4: how to use top.bit file on basys 3 board

Results

1. Part A waveforms

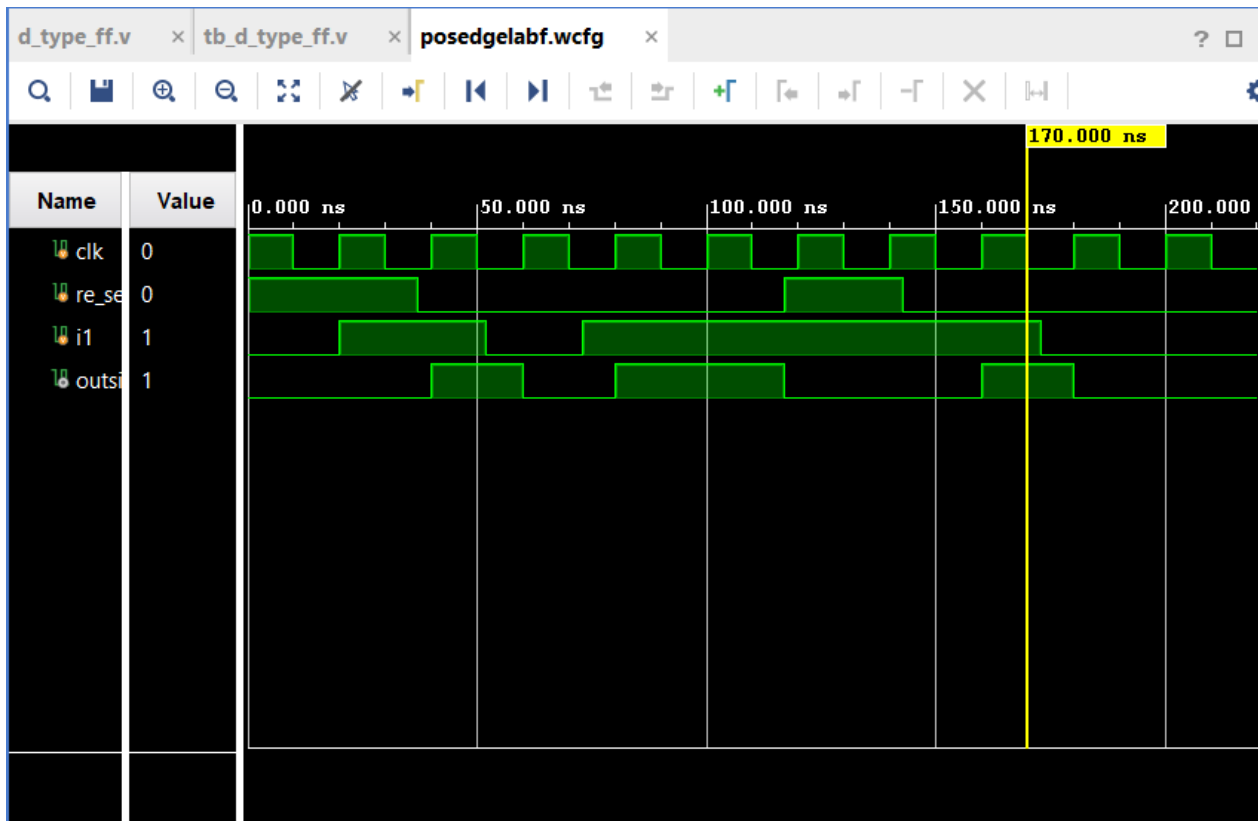


Figure 5: Asynchronous inputs for posedge clock

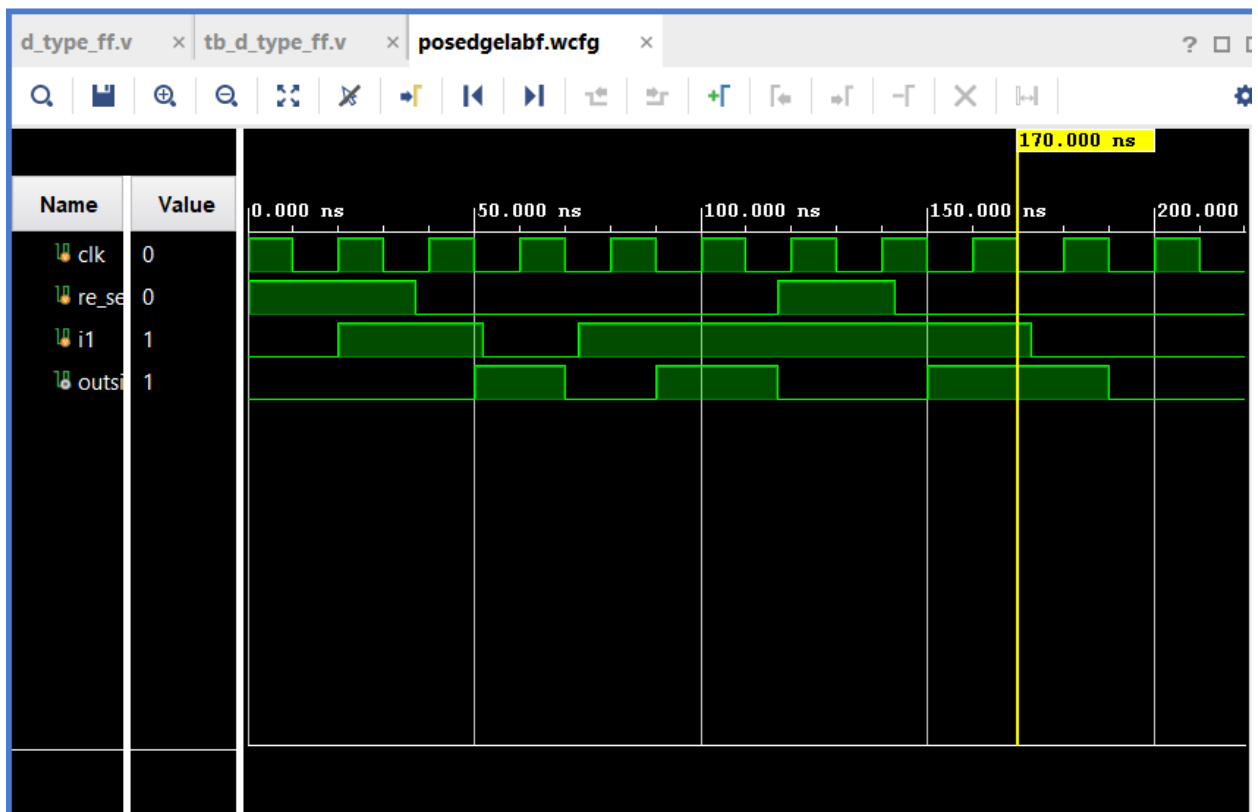


Figure 6: negedge asynchronous

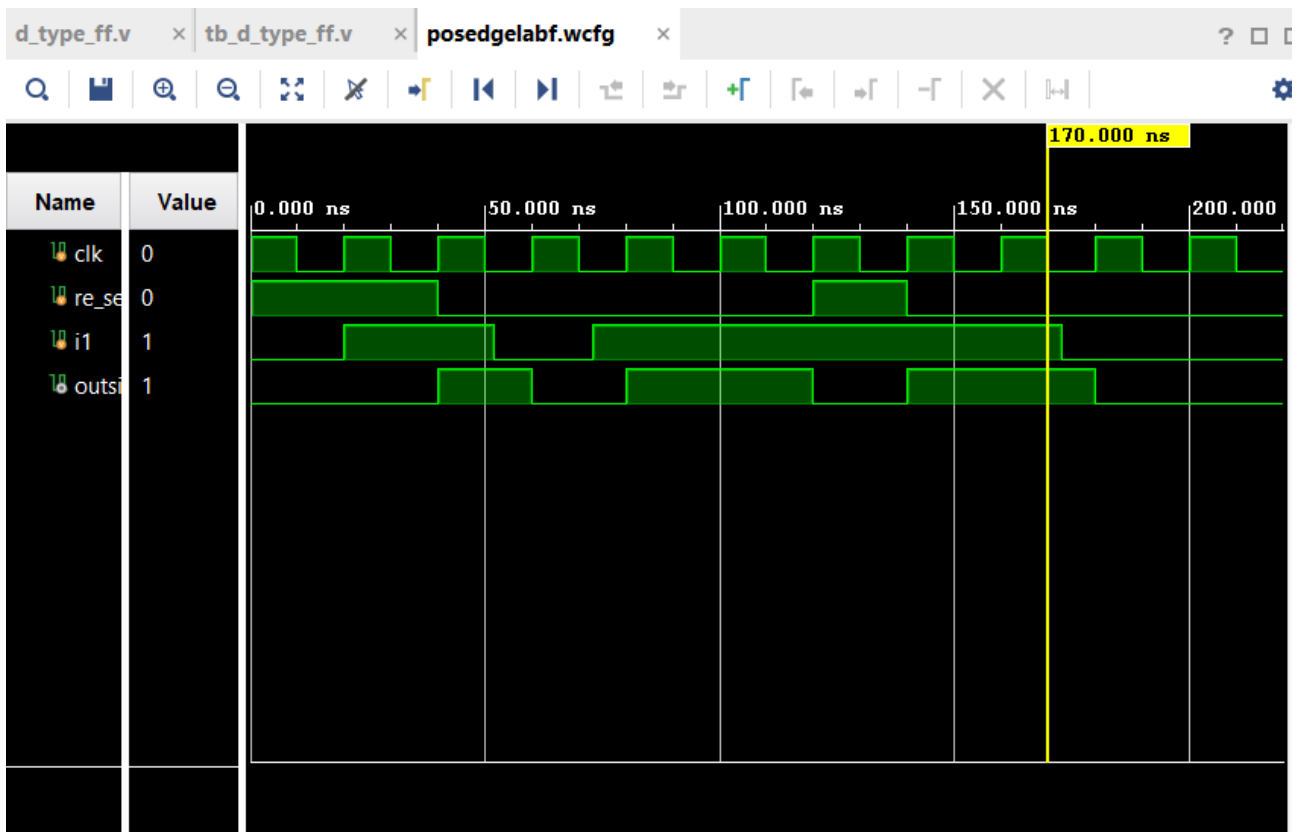


Figure 7: Synchronous inputs for posedge clock

2. Part B basys 3 implemented design

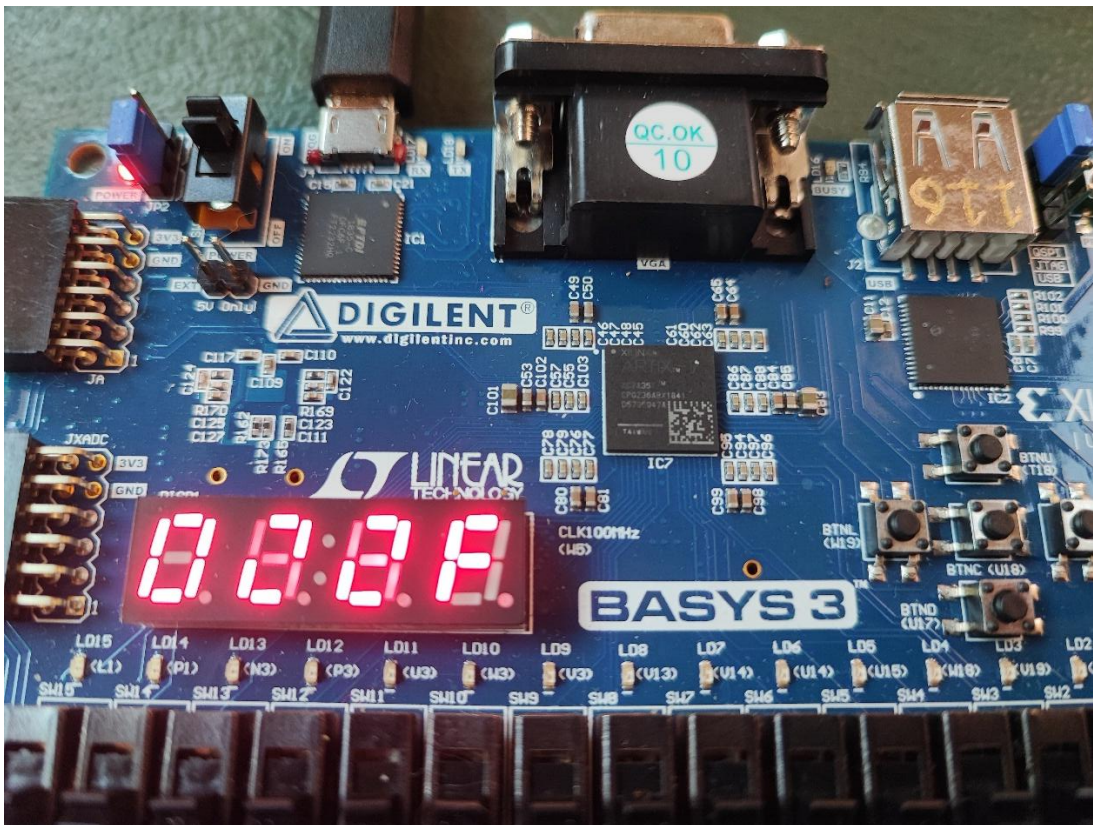


Figure 8: Working design of temperature

3. Part C waveforms

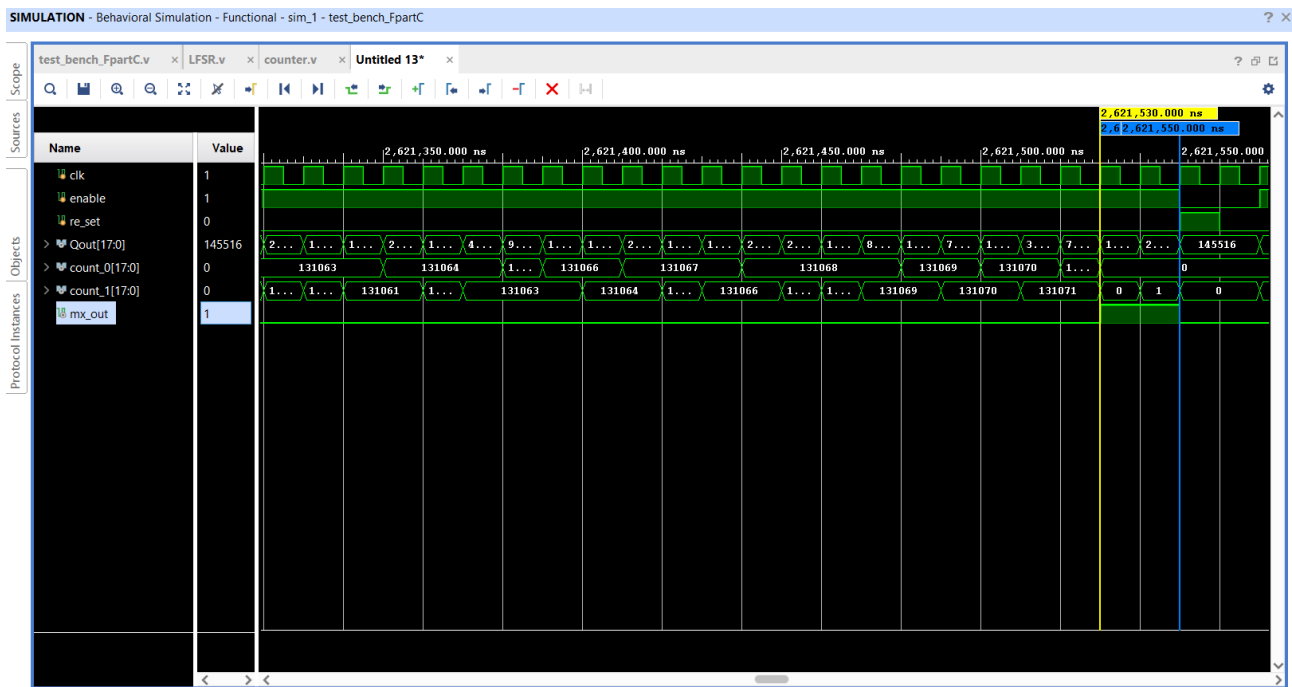


Figure 9: Proper seed value waveform

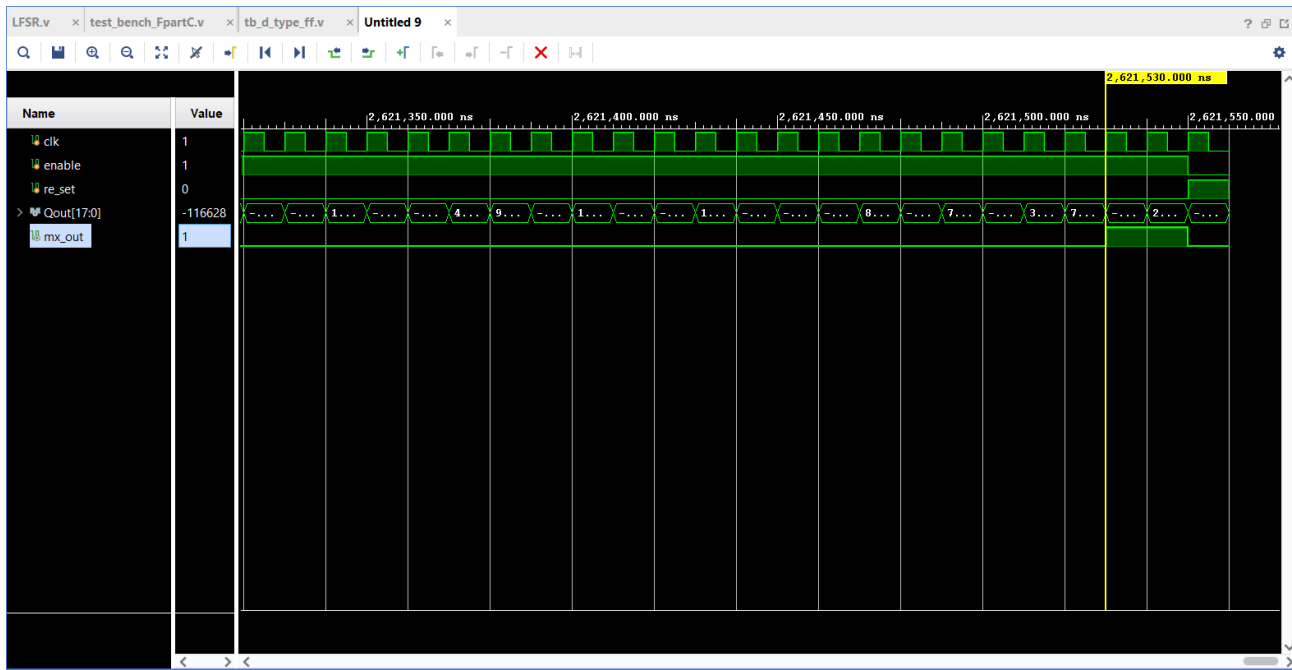


Figure 10: Close up of proper seed value before reset

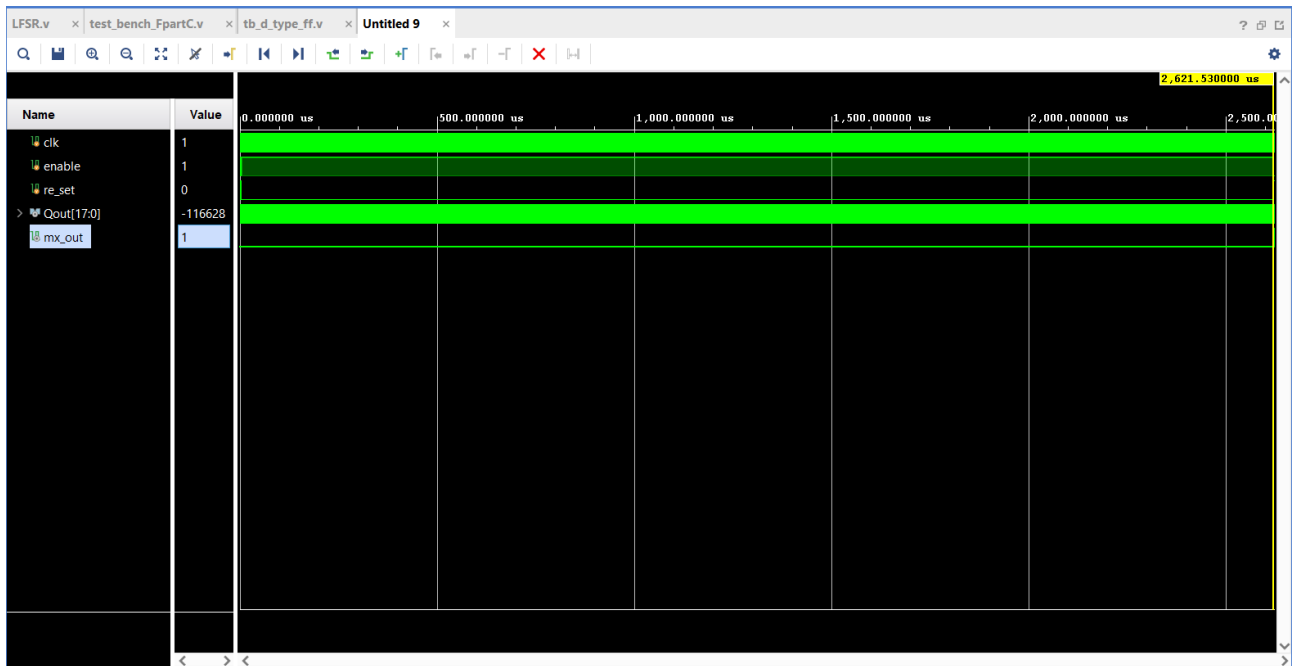


Figure 11: forbidden seed value waveform

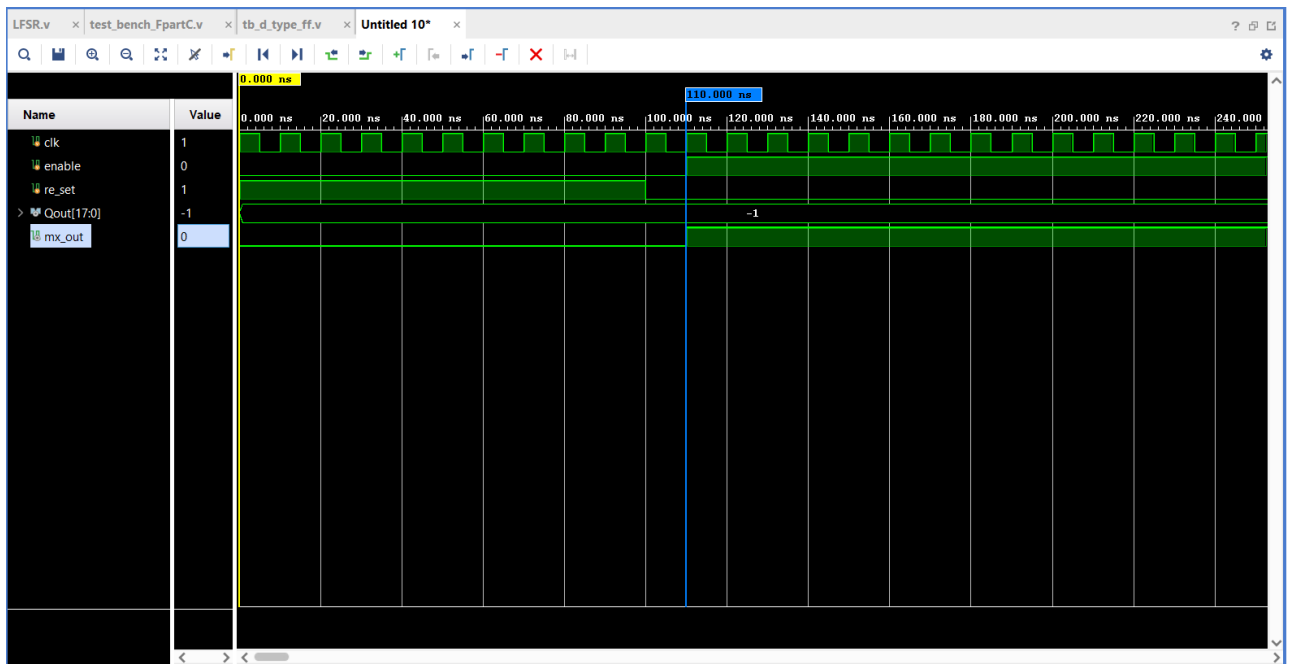


Figure 12: forbidden seed value waveform close up after reset

4. Part C advanced

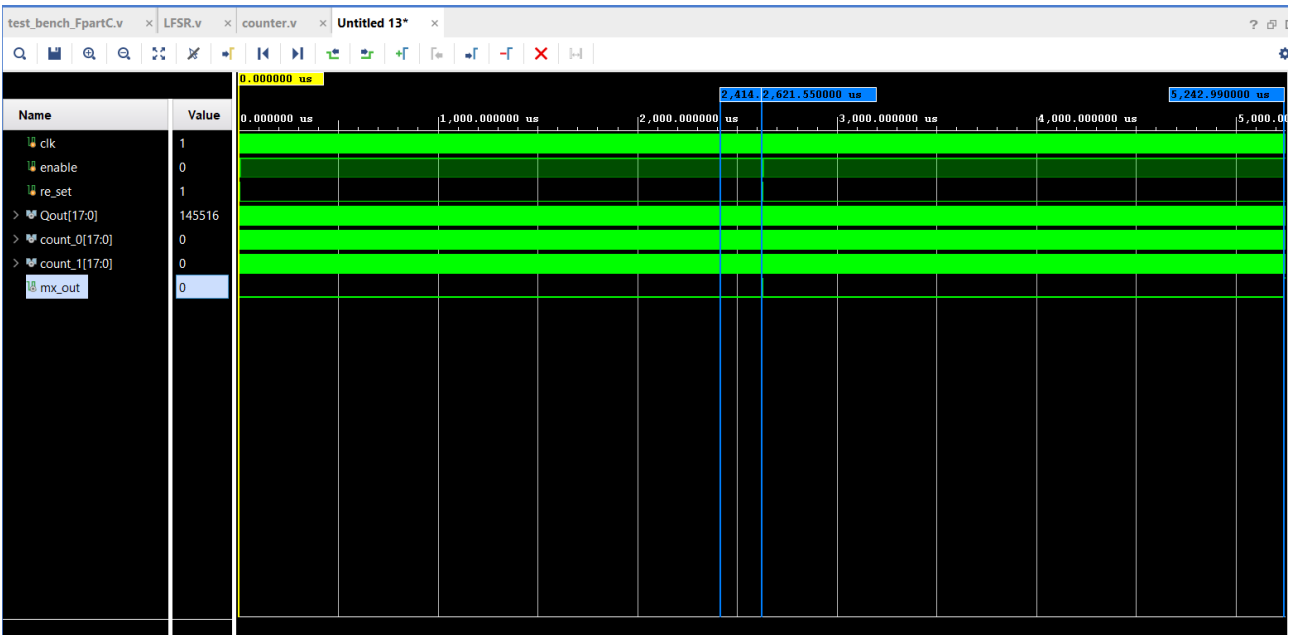


Figure 13: Waveform of LFSR with counter from afar

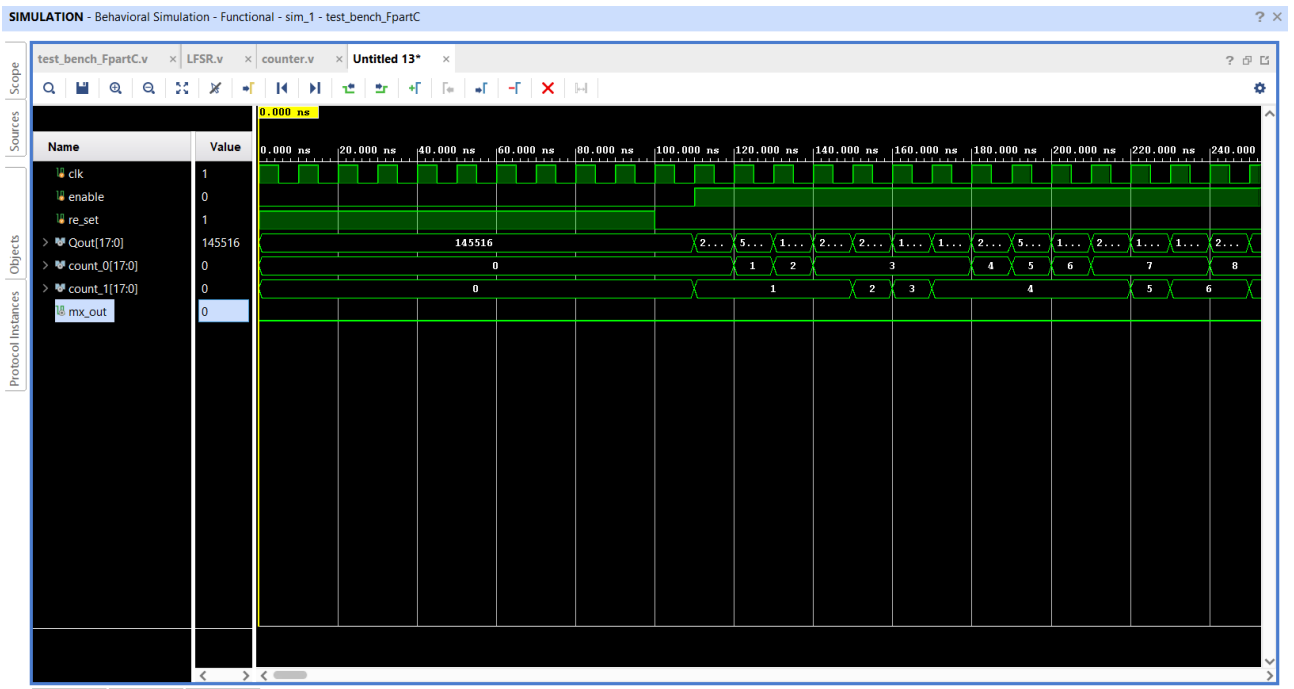


Figure 14: Initial reset value of LFSR with counter

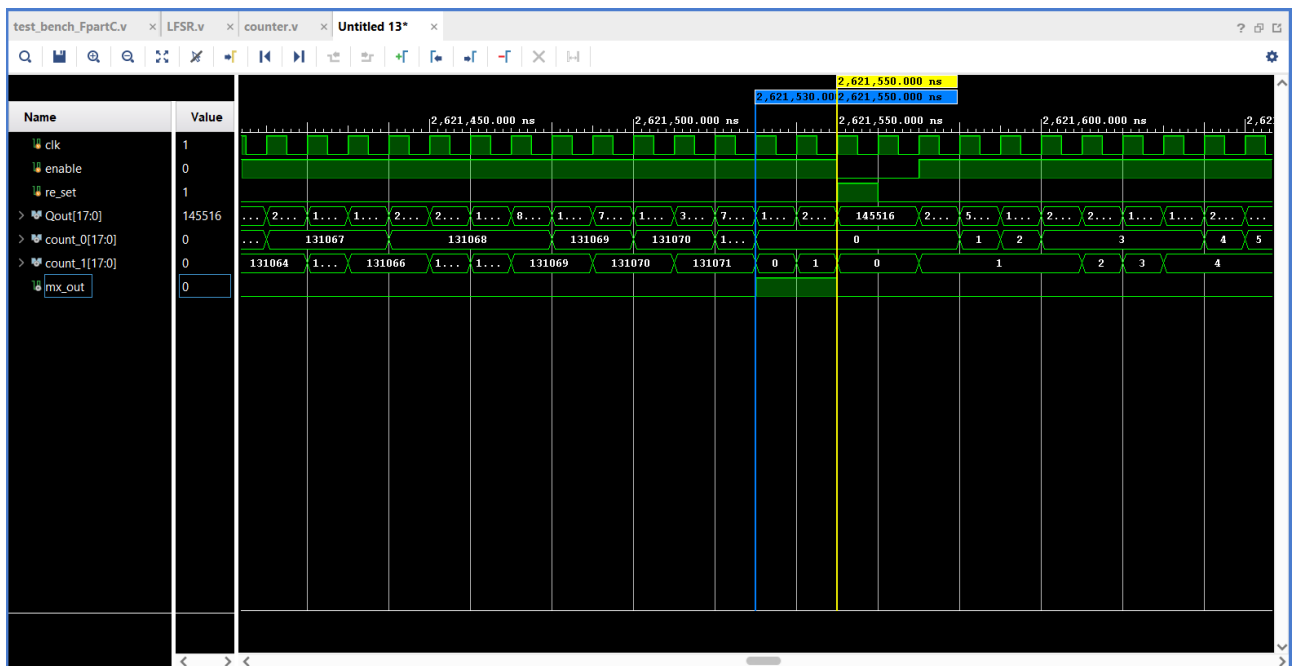


Figure 15: LFSR with counter reaching max val before resetting

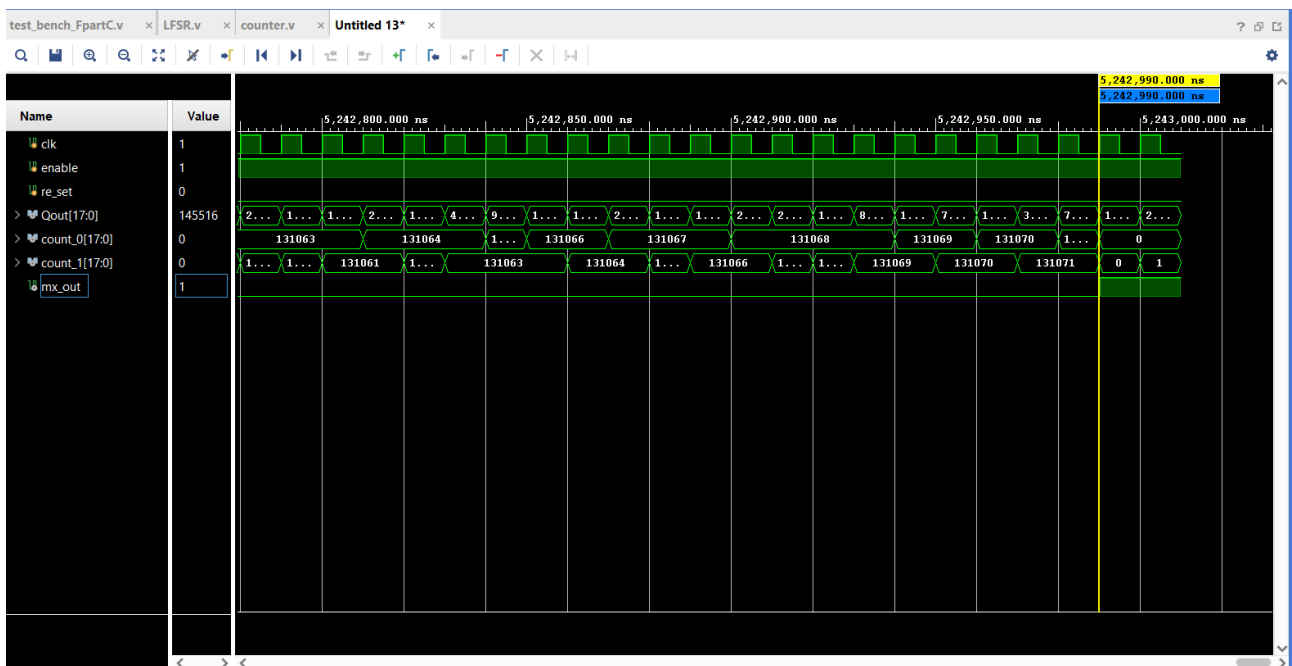


Figure 16: Reaching max val again to make sure counter is reset properly

Discussion

The difference between the negative edge of clock activated circuit vs the positive edge of clock activated circuit waveform is that the output value changes according to the negative edge of the clock instead of the positive edge of the clock. A synchronous reset also changes the waveform as a synchronous reset of a full period of the clock resets twice the output instead of once.

The implemented design works like expected. As when the centre button is pressed the value assigned to the memory is 22. Any other button pressed either decreases or increases and the left most switch resets the value in the memory to 0.

I guessed that when counting the 1's and 0's outputted by the LFSR I expected it to be $2^n/2$ per 1's outputted and the same for 0's. I expected this as if the LFSR iterates different combination of numbers up to 2^n-1 it must mean that it goes through all possible combination except an all 1 array. When it fully loops it will have outputted 2^n bits which 50% must be 1 and 50% must be 0.